

Chapter – 4

Data Encryption Standard (DES)

Outline

- Introduction
- Feistel Cipher Structure
- The Data Encryption Standard (DES)
- DES Analysis
- The strength of DES
- Multiple Encryption With DES
- Differential and Linear Cryptanalysis

1. Introduction to DES

- DES was published by US National Bureau of Standards (NBS) in 1977 which was the predecessor of the US National Institute of Standards and Technology (NIST).
- DES was based on the Lucifer encryption algorithm developed by an IBM research group led by Horst Feistel.
- DES was widely used from mid-1970's to early-2000's.
- Although gradually phasing out, DES played an important role in modern cryptography and represents a popular design paradigm for data encryption.
- DES's encryption and decryption structures are symmetrical, and they use 4 basic operations: *exclusive-OR, permutation, substitution, and circular shift.*

Cont...

- *Exclusive-OR,*

- ✓ $X \oplus 0^n = X, \quad X \oplus X = 0^n, \quad X \oplus Y = Y \oplus X$

- *Permutation (Transposition)*

- ✓ No elements are added/deleted/replaced in the sequence, rather order in which elements appear in the sequence is changed

- *Substitution*

- ✓ Each plaintext element/group of elements is uniquely replaced by corresponding ciphertext element or group of elements

- *Circular shift*

- It shifts each bit in an n-bit word K positions to the left and the leftmost k bits become the original rightmost bits.

Cont...

Goals of DES

- If frequency distribution of the various letters or words or phrases are in any way reflected in the ciphertext, the cryptanalyst may be able to deduce the encryption key, part of the key, or at least a set of keys likely to contain the exact key.
- **Diffusion**
 - *Seeks to make the statistical relationship between the plaintext and ciphertext as complex as possible in order to thwart attempts to deduce the key*
- **Confusion**
 - *Seeks to make relationship between the statistics of ciphertext and the value of encryption key as complex as possible.*

Feistel Cipher Structure (FCS)

- The inputs to the encryption algorithm are a plaintext block of length $2l$ bits and a key K .
- Plaintext block is divided into two halves, LE_0 and RE_0
- FCS only uses basic operations of
 - *XOR and*
 - *Substitution*
- The two halves of the data pass through n rounds of processing and then combine to produce the ciphertext block.
- Each round i has as inputs LE_{i-1} and RE_{i-1} derived from the previous round, as well as a subkey K_i derived from the overall K .
- In general, the subkeys K_i are different from K and from each other

Cont...

Algorithm Description

- Elements of FCS
 - ✓ Plaintext
 - ✓ Secret key and subkeys
 - ✓ Substitution Function
 - ✓ XOR operations
 - ✓ Round number
- Assume
 - ✓ n be a positive integer
 - ✓ F denote substitution function
 - ✓ K denotes the encryption key

Cont . . .

Algorithm Description

- **Step 1:** FCS first generates n subkeys of a fixed length from the encryption key K . Let these subkeys be $K_1, K_2, \dots, K_n$
- **Step 2:** Divide a $2l$ -bit plaintext block M into two halves LE_0 and RE_0 of equal length, where LE_0 and RE_0 , respectively, the prefix substring and the suffix substring of M .
- **Step 3:** a substitution function F , takes an l -bit suffix substring (RE_i where $i = 1, 2, \dots, n$) of M and a subkey as Input, and generates an l -bit binary string as output
- **Step 4:** Making XOR of the output of substitution function and the left half of the data (LE_i where $i = 1, 2, \dots, n$)

Cont . . .

Algorithm Description

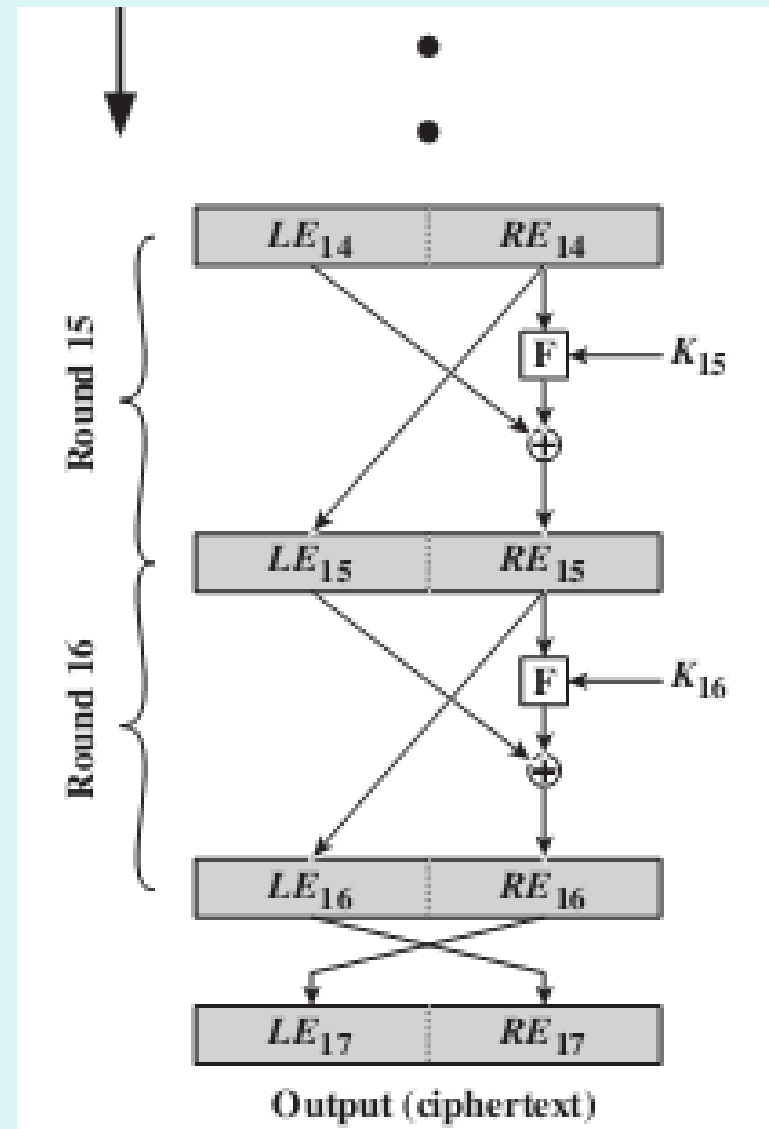
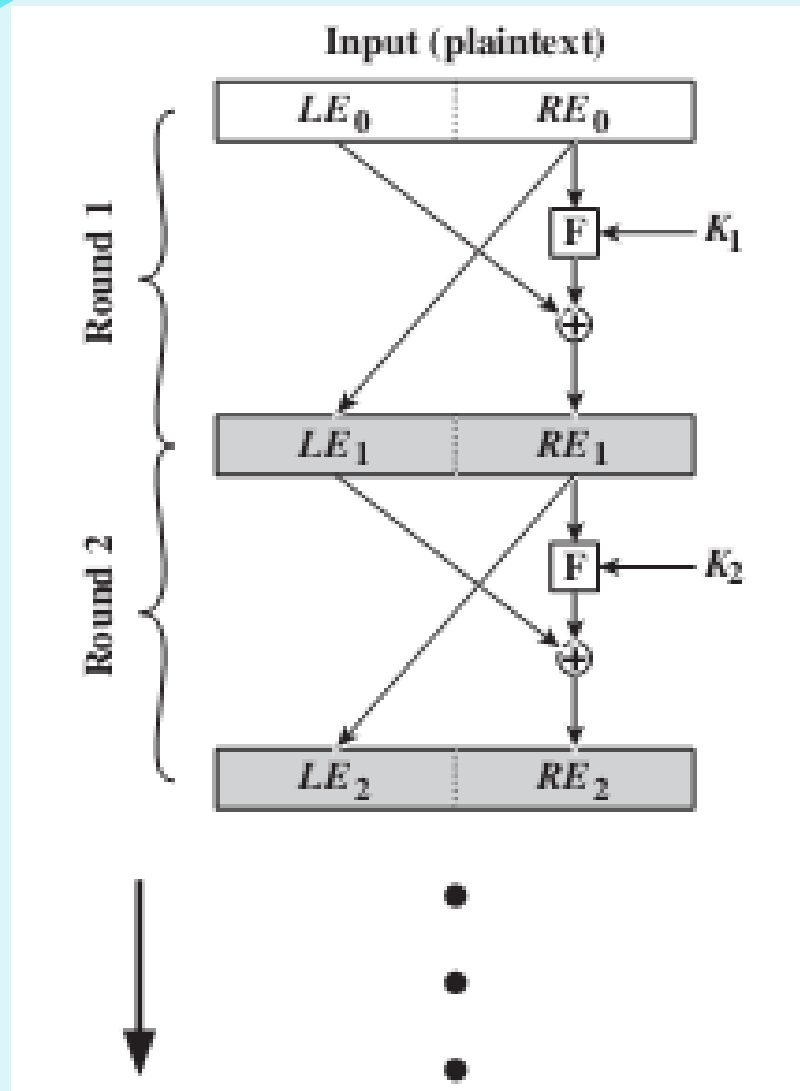
- **Step 5:** increment i and $LE_i = RE_{i-1}$ and $RE_i = \text{Output of step 4}$

$$L_i = R_{i-1},$$

$$R_i = L_{i-1} \oplus F(R_{i-1}, K_i).$$

- **Step 6:** Repeat step 3 through step 5 until round is over (n times)
- **Step 7:** interchange of the two halves of the data of final round (permutation)
- After n rounds, the plaintext block $M = LE_0 RE_0$ is transformed to $C = LE_n RE_n$

Cont...



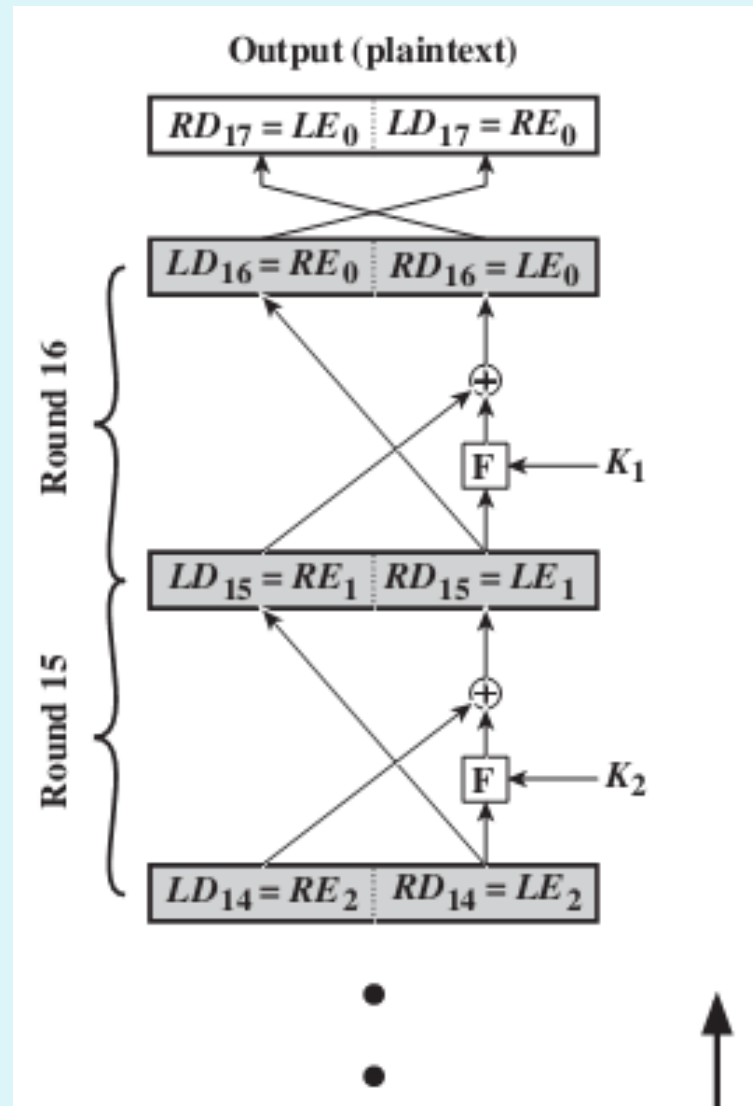
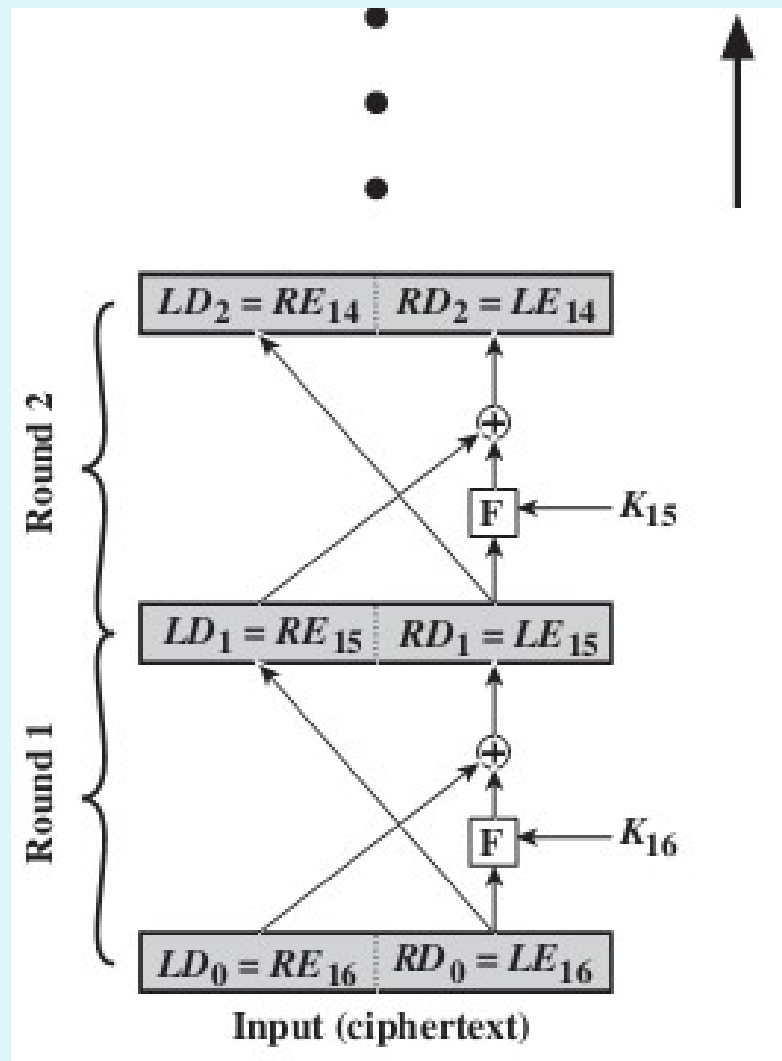
Feistel Encryption (16 rounds)

Cont . . .

FCS Decryption

- FCS decryption algorithm is symmetrical to the FCS encryption algorithm, except that the subkeys are applied in the reverse order.
- The rule is as follows:
 - *Use the ciphertext as input to the algorithm, but use the subkeys in reverse order. That is, use K_n in the first round, K_{n-1} in the second round, and so on, until K_1 is used in the last round*

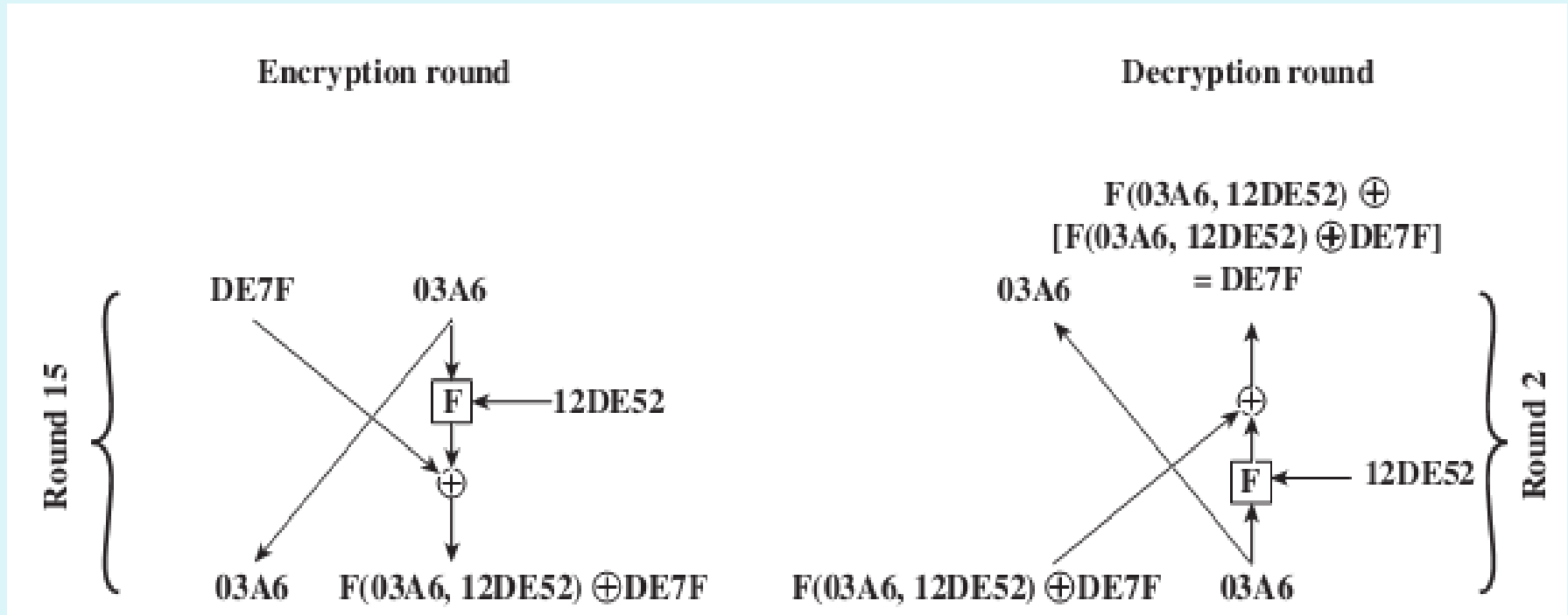
Cont...



Feistel Decryption (16 rounds)

Cont...

Example of FCS



- To help clarify the preceding concepts, let us look at a specific example and focus on the 15th round of encryption, corresponding to the second round of decryption

Cont...

- Suppose that the blocks at each stage are 32 bits (two 16-bit halves) and that the key size is 24 bits.
- Suppose that at the end of encryption round fourteen, the value of the intermediate block (in hexadecimal) is DE7F03A6.
- Then $LE_{14} = \text{DE7F}$ and $RE_{14} = \text{03A6}$
- Also assume that the value of $K_{15} = \text{12DE52}$.
- After round 15, we have
 $LE_{15} = \text{03A6}$ and $RE_{15} = \text{F(03A6, 12DE52)} \oplus \text{DE7F}$.

Now let's look at the decryption. We assume that $LD_1 = RE_{15}$ and $RD_1 = LE_{15}$, as shown in Figure 3.3, and we want to demonstrate that $LD_2 = RE_{14}$ and $RD_2 = LE_{14}$. So, we start with $LD_1 = \text{F(03A6, 12DE52)} \oplus \text{DE7F}$ and $RD_1 = \text{03A6}$. Then, from Figure 3.3, $LD_2 = \text{03A6} = RE_{14}$ and $RD_2 = \text{F(03A6, 12DE52)} \oplus [\text{F(03A6, 12DE52)} \oplus \text{DE7F}] = \text{DE7F} = LE_{14}$.

FCs design elements

- The exact realization of a Feistel network depends on the choice of the following parameters and design features
 - Block size
 - Key size
 - Number of rounds
 - Subkey generation algorithm
 - Round function
 - Fast software en/decryption
 - Ease of analysis

The DES Algorithm

The DES Cipher

- DES is a block cipher, operating on blocks of *64 bits (8-16 characters, depending on encoding)*
- Key is any 56 bit value, generally expressed as a 64 bit number, with every *8th bit* acting as a parity bit.
- Encryption is accomplished by alternating *substitutions and permutations*, performed in a series of "*rounds*" where each round employs a different *subkey* derived from the original key.
- Algorithm is based on the idea of a "*Feistel Network*", which provides a basis for generating mapping functions that are guaranteed to be invertible, and their own inverses.

Cont...

- Basically, a bit string is split into two halves (L, R), some function f is applied to the right half to obtain $f(R)$, and a new string $(L \text{ XOR } f(R), R)$ is produced.
- Clearly, if this operation is applied twice, we get the original string back $(L \text{ XOR } f(R) \text{ XOR } f(R), R) = (L, R)$.
- DES employs a series of functions $f1, f2 \dots f16$, and swaps the L and R halves after each application except the last one.
- So (L, R) goes to $(R, L \text{ XOR } f(R))$. This ensures that both left and right halves get mixed up.
- The particular functions depend on the key.
- Each such operation is called round, so DES uses 16 rounds.

Cont...

Algorithm

Step 1: The 64 bit plain text blocks are handed over to the *initial permutation (IP) function*.

Step 2: Perform an initial permutation (IP) of the plaintext, the block is broken into left and right halves (LPT, RPT) 32-bit each.

Step 3: Perform 16 rounds of encryption process each round with its own key.

For each of the 16 rounds

$$LPT_i = RPT_{i-1} \text{ and}$$

$$RPT_i = LPT_{i-1} \oplus F(RPT_{i-1}, K_i)$$

Cont...

- Rounds are defined as follows in the algorithm:

Step 3a: Key transformation

Step 3b: Expansion Permutation (EP)

Step 3c: S-Box Substitution

Step 3d: P-Box Permutation

Step 3e: XOR (*LPT* and Output of *step 3d*)

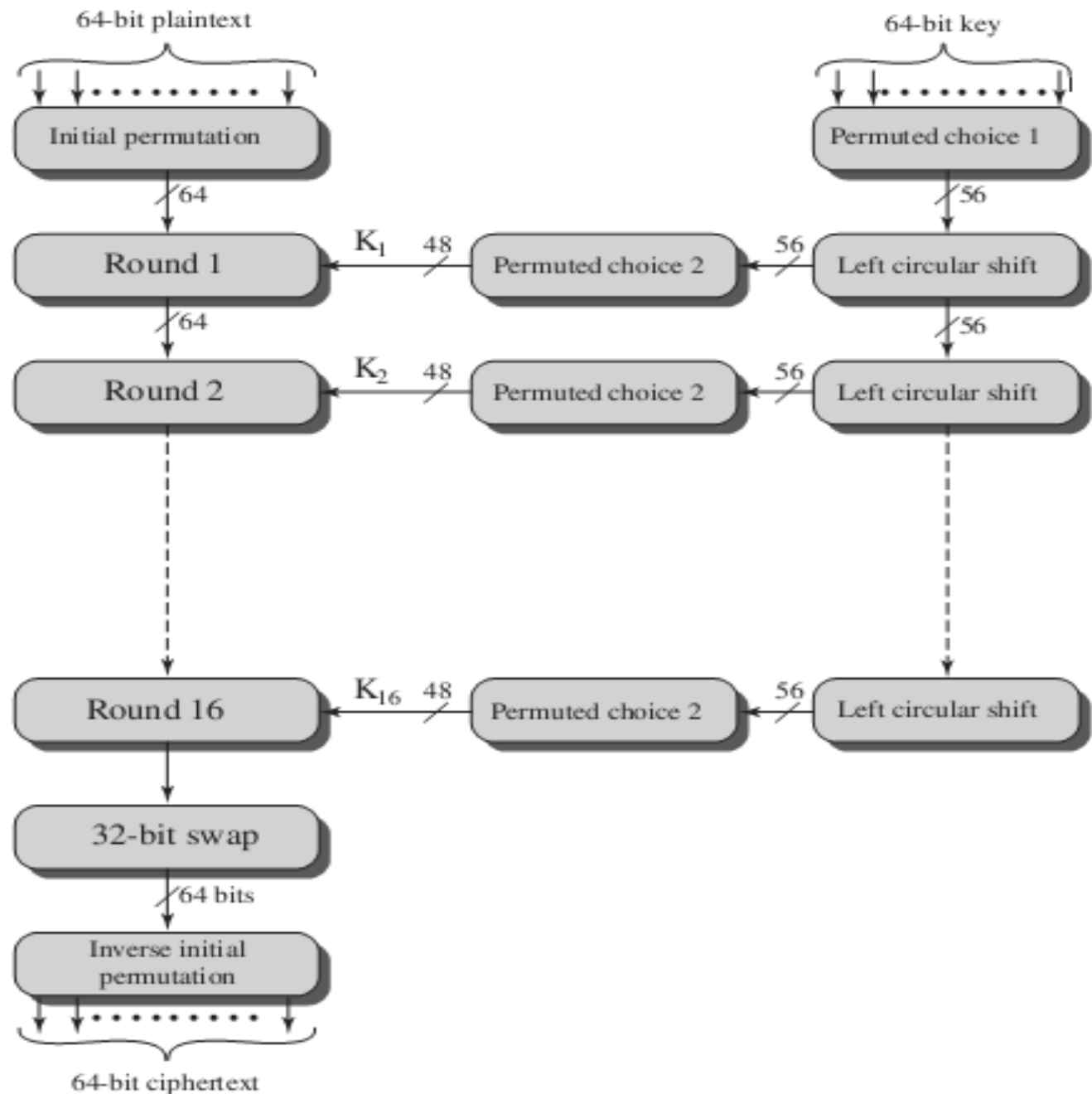


Next Round ----> $LPT_i = RPT_{i-1}$ and $RPT_i = \text{Output of step 3e}$

Step 4: The output of the final round (*LPT* and *RPT*) are **Swapped and rejoined**, then sent through a final permutation, which is the inverse of the initial permutation. This is final result of encrypting the block.

Step 5: The result of this process produces 64-bit cipher text.

Cont...



Cont...

Initial and Final Permutation

- The initial permutation (IP) and its inverse (final permutation) IP^{-1} rearranges bits according to the following table, where the entries in the table show the new arrangement of bits from their initial order.
- They defined/presented as a table for ease of presentation as follow.

(a) Initial Permutation (IP)

58	50	42	34	26	18	10	2
60	52	44	36	28	20	12	4
62	54	46	38	30	22	14	6
64	56	48	40	32	24	16	8
57	49	41	33	25	17	9	1
59	51	43	35	27	19	11	3
61	53	45	37	29	21	13	5
63	55	47	39	31	23	15	7

Cont...

(b) Inverse Initial Permutation (IP^{-1})

40	8	48	16	56	24	64	32
39	7	47	15	55	23	63	31
38	6	46	14	54	22	62	30
37	5	45	13	53	21	61	29
36	4	44	12	52	20	60	28
35	3	43	11	51	19	59	27
34	2	42	10	50	18	58	26
33	1	41	9	49	17	57	25

- Both the tables specifies the input permutation on a 64-bit block and interpreted in the same fashion .
- Example meaning of IP table:** the first bit of the output is taken from the 58th bit of the input; the second bit from the 50th bit, and so on, with the last bit of the output taken from the 7th bit of the input.
- Aimed at optimizing early hardware implementations and they have no cryptography significance in DES

Cont...

Key Transformation

- DES operates on the 64-bit blocks using *key* sizes of 56- bits.
- The keys are actually stored as being 64 bits long, but every 8th bit in key is not used (i.e. bits numbered 8, 16, 24, 32, 40, 48, 56, 64).
- The 64-bit key is permuted according to the following table, *Permuted choice one (PC-1)*
- *And this permutation performed only once at the beginning, before encryption round entered.*
 - ✓ Since the 1st entry in the table is "57", this means that the 57th bit of the original key **K** becomes first bit of the permuted key **K+**.
 - ✓ The 49th bit of original key becomes second bit of permuted key. The 4th bit of the original key is the last bit of the permuted key.

Cont...

(b) Permuted Choice One (PC-1)

57	49	41	33	25	17	9
1	58	50	42	34	26	18
10	2	59	51	43	35	27
19	11	3	60	52	44	36
63	55	47	39	31	23	15
7	62	54	46	38	30	22
14	6	61	53	45	37	29
21	13	5	28	20	12	4

➤ **Note** only 56 bits of the original key appear in the permuted key.

Key Generation

- ✓ The round-key generator creates sixteen 48-bit keys out of a 56-bit cipher key.

Cont...

- ✓ The 56 key bits are shifted, in two halves, by one or two bits, and 48 bits selected by a "compression permutation" called *Permuted choice two (PC-2)*, to form a sub-key for the round.

Shifting

Rounds	Shift
1, 2, 9, 16	one bit
Others	two bits

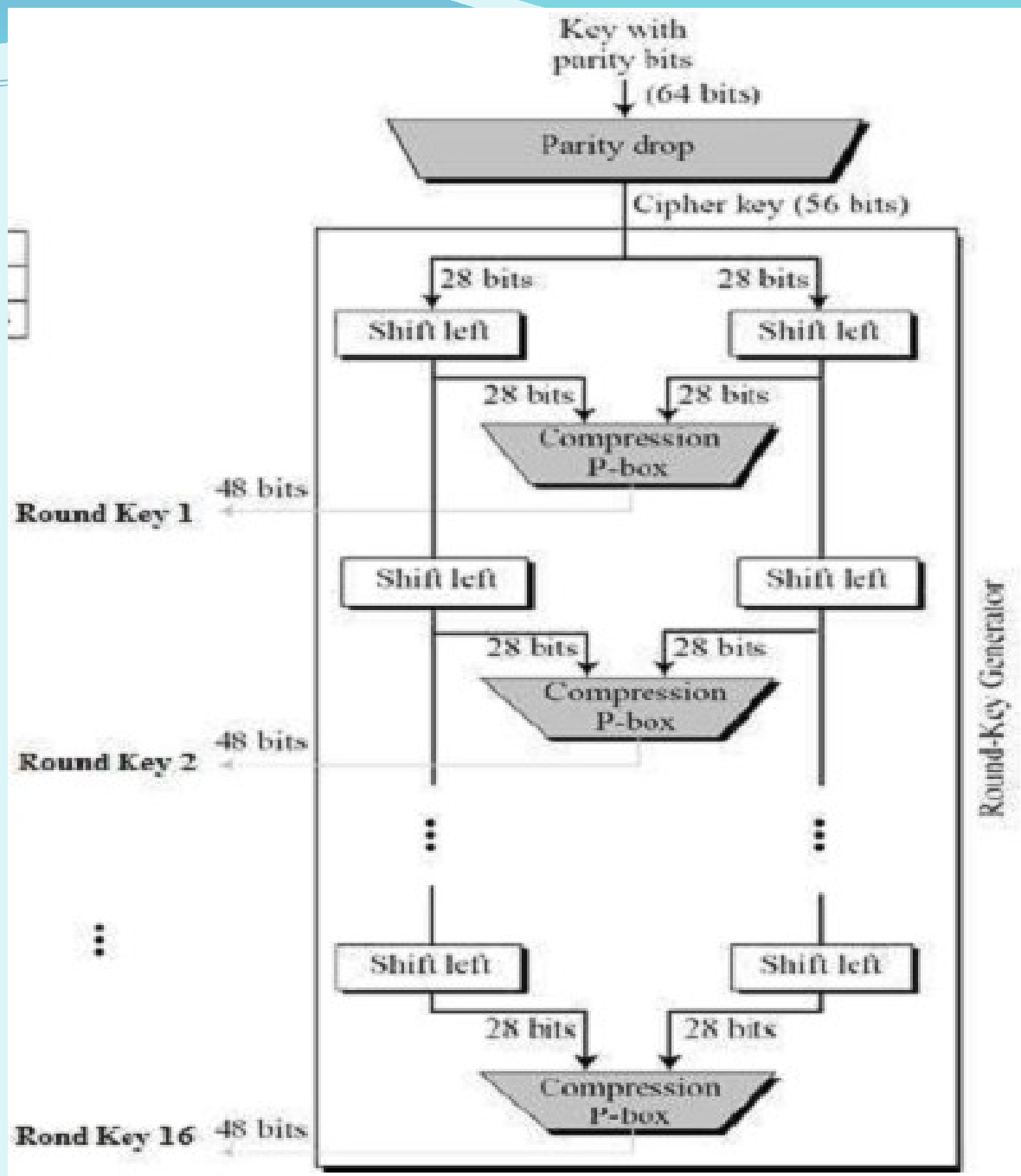
(c) Permuted Choice Two (PC-2)

14	17	11	24	1	5	3	28
15	6	21	10	23	19	12	4
26	8	16	7	27	20	13	2
41	52	31	37	47	55	30	40
51	45	33	48	44	49	39	56
34	53	46	42	50	36	29	32

Round Number	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Bits Rotated	1	1	2	2	2	2	2	2	1	2	2	2	2	2	2	1

- ✓ The process of key generation is depicted in the following illustration

Cont...



Cont...

Example of Key Transformation

- From the original 64-bit key

$\mathbf{K} = 00010011\ 00110100\ 01010111\ 01111001\ 10011011$
 $10111100\ 11011111\ 11110001$

- We get the 56-bit permutation

$\mathbf{K}^+ = 1111000\ 0110011\ 0010101\ 0101111\ 0101010\ 1011001$
 $1001111\ 0001111$

- Next, split this key into left and right halves, $\mathbf{C}_0$ and $\mathbf{D}_0$, where each half has 28 bits. So that from the permuted key $\mathbf{K}^+$, we get

$\mathbf{C}_0 = 1111000\ 0110011\ 0010101\ 0101111$

$\mathbf{D}_0 = 0101010\ 1011001\ 1001111\ 0001111$

- With $\mathbf{C}_0$ and $\mathbf{D}_0$ defined, we now create sixteen blocks $\mathbf{C}_n$ and $\mathbf{D}_n$, $1 \leq n \leq 16$. Each pair of blocks $\mathbf{C}_n$ and $\mathbf{D}_n$ is formed from the previous pair $\mathbf{C}_{n-1}$ and $\mathbf{D}_{n-1}$, respectively, for $n = 1, 2, \dots, 16$, through "left shifts" of the previous block according shift schedule.

Cont...

- This means, for example, C_3 and D_3 are obtained from C_2 and D_2 , respectively, by two left shifts, and C_{16} and D_{16} are obtained from C_{15} and D_{15} , respectively, by one left shift.
- In all cases, by a single left shift is meant a rotation of the bits one place to the left, so that after one left shift the bits in the 28 positions are the bits that were previously in positions 2, 3,..., 28, 1.
- So that from original pair C_0 and D_0 we obtain:

$$C_0 = 1111000011001100101010101111$$

$$D_0 = 0101010101100110011110001111$$

$$C_1 = 1110000110011001010101011111$$

$$D_1 = 1010101011001100111100011110$$

$$C_2 = 1100001100110010101010111111$$

$$D_2 = 0101010110011001111000111101$$

Note:
Similarly
Compute for the
others also

Cont . . .

- We now form the keys K_n , for $1 \leq n \leq 16$, by applying the permutation using **PC-2** table to each of concatenated pairs $C_n D_n$.
- Each pair has 56 bits, but **PC-2** only uses 48 of these.
- Accordingly for the first key we have

$$C_1 D_1 = 1110000 \ 1100110 \ 0101010 \ 1011111 \ 1010101 \ 0110011 \\ 0011110 \ 0011110$$

- Which, after we apply the permutation **PC-2**, becomes

$$K_1 = 000110 \ 110000 \ 001011 \ 101111 \ 111111 \ 000111 \ 000001 \ 110010$$

- For the other keys (K_2, K_3, K_4) we have

$$K_2 = 011110 \ 011010 \ 111011 \ 011001 \ 110110 \ 111100 \ 100111 \ 100101$$

$$K_3 = 010101 \ 011111 \ 110010 \ 001010 \ 010000 \ 101100 \ 111110 \ 011001$$

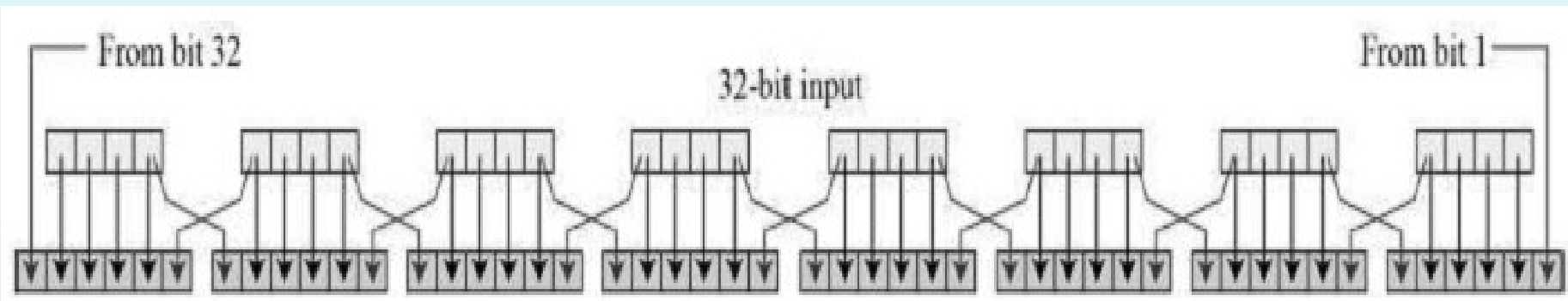
$$K_4 = 011100 \ 101010 \ 110111 \ 010110 \ 110110 \ 110011 \ 010100 \ 011101$$

Note: Similarly Compute for the others also

Cont...

Expansion Permutation (EP)

- Since right input is 32-bit and round key is a 48-bit, we first need to expand right input to 48 bits using an "expansion permutation".
- First the 32-bit text is divided into 8 blocks of 4 bits each.
- Then by adding 2 bits extra, that is the first bit of the block 1 is the last bit of the block 8 and the last bit of the block 8 is the first bit of the 7th block the 48-bit text is obtained.
- Permutation logic is graphically depicted in the



Cont...

- The graphically depicted permutation logic is generally described as table in DES specification as follow.

32	01	02	03	04	05
04	05	06	07	08	09
08	09	10	11	12	13
12	13	14	15	16	17
16	17	18	19	20	21
20	21	22	23	24	25
24	25	26	27	28	29
28	29	31	31	32	01

- Example:** We calculate $E(R_0)$ from R_0 as follows:

$$R_0 = 1111\ 0000\ 1010\ 1010\ 1111\ 0000\ 1010\ 1010$$

$$E(R_0) = 011110\ 100001\ 010101\ 010101\ 011110\ 100001$$

$$010101\ 010101$$

Cont...

- After the expansion permutation, DES does XOR operation on the expanded right section and the round key. The round key is used only in this operation.
- Example:** For K_I , $E(R_o)$, we have

$$K_I = \begin{array}{cccccccc} 000110 & 110000 & 001011 & 101111 & 111111 & 000111 & 000001 & \\ & 110010 & & & & & & \end{array}$$

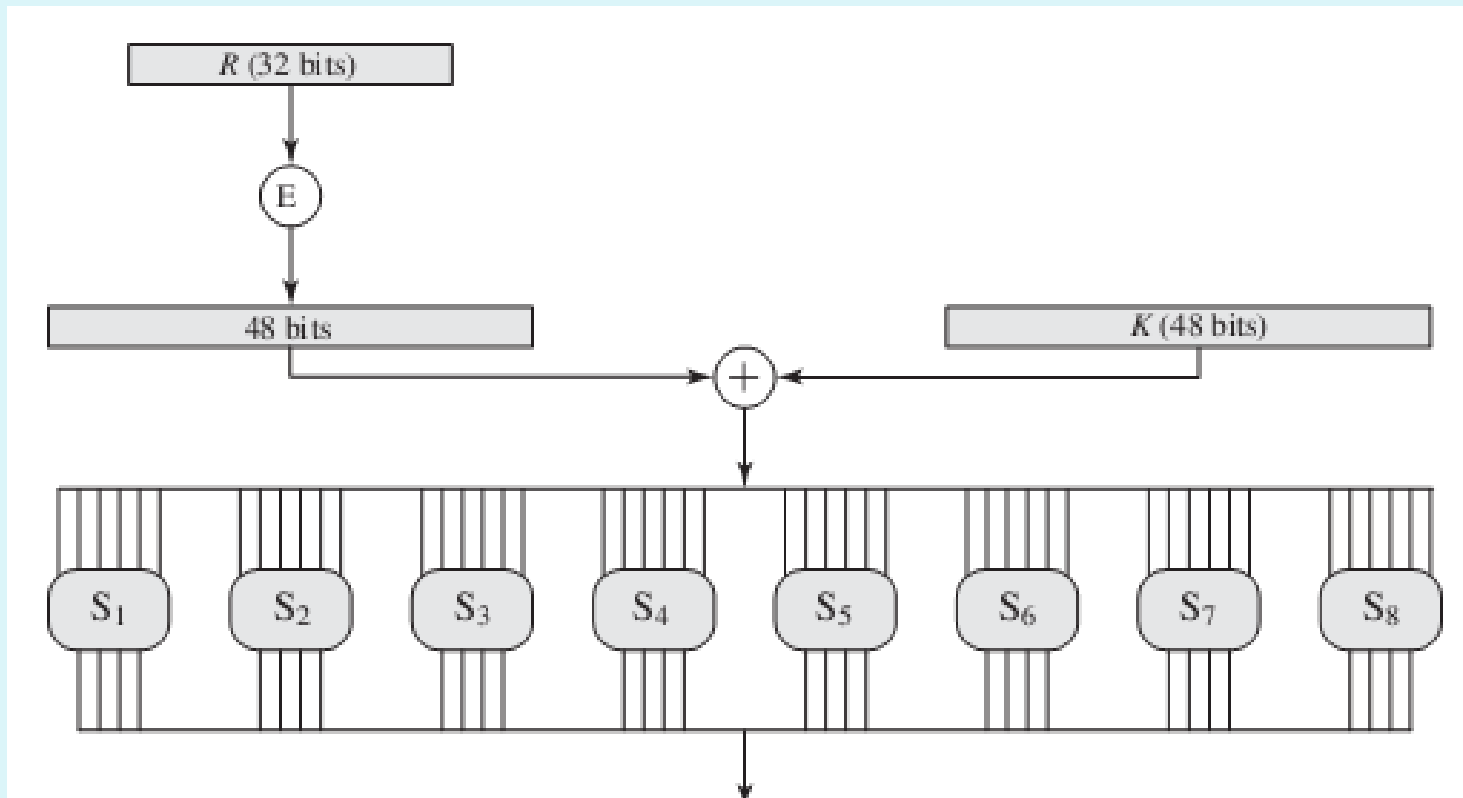
$$E(R_o) = \begin{array}{cccccccc} 011110 & 100001 & 010101 & 010101 & 011110 & 100001 & & \\ 010101 & 010101 & & & & & & \end{array}$$

$$K_I + E(R_o) = \begin{array}{cccccccc} 011000 & 010001 & 011110 & 111010 & 100001 & 100110 & & \\ 010100 & 100111 & & & & & & \end{array}$$

Cont...

S-Box Substitution

- The result of Expansion permutation which XORed with Key is sent through a set of 8 fixed "S-boxes"
- The S-boxes carry out the real mixing confusion.



Cont...

- DES uses 8 S-boxes, each with a 6-bit input and a 4-bit output
- The output of all eight s-boxes is then combined in to 32 bit section.

How it works

- ✓ The 48-bit blocks of S-box input divided into 8 blocks of 6 bits each.
- ✓ Decimal equivalent of the first and last bit in a block denotes the row number and decimal equivalent of the bit 2,3,4 and 5 denotes the column number of the S-Box Substitution table.
- ✓ Check the value and take the binary equivalent of the number.
- ✓ The result is 4-bit binary number
- Write the previous result, which is 48 bits, in the form:
$$K_n + E(R_{n-1}) = B_1B_2B_3B_4B_5B_6B_7B_8, \text{ where } B_i \text{ is a group of six bits.}$$
- We now calculate:- $S_1(B_1)S_2(B_2)S_3(B_3)S_4(B_4)S_5(B_5)S_6(B_6)S_7(B_7)S_8(B_8)$
where $S_i(B_i)$ refers to the output of the i -th S box.

Cont . . .

Tables defining the functions $S_1, \dots, S_4$

S_1

14	4	13	1	2	15	11	8	3	10	6	12	5	9	0	7
0	15	7	4	14	2	13	1	10	6	12	11	9	5	3	8
4	1	14	8	13	6	2	11	15	12	9	7	3	10	5	0
15	12	8	2	4	9	1	7	5	11	3	14	10	0	6	13

S_2

15	1	8	14	6	11	3	4	9	7	2	13	12	0	5	10
3	13	4	7	15	2	8	14	12	0	1	10	6	9	11	5
0	14	7	11	10	4	13	1	5	8	12	6	9	3	2	15
13	8	10	1	3	15	4	2	11	6	7	12	0	5	14	9

S_3

10	0	9	14	6	3	15	5	1	13	12	7	11	4	2	8
13	7	0	9	3	4	6	10	2	8	5	14	12	11	15	1
13	6	4	9	8	15	3	0	11	1	2	12	5	10	14	7
1	10	13	0	6	9	8	7	4	15	14	3	11	5	2	12

S_4

7	13	14	3	0	6	9	10	1	2	8	5	11	12	4	15
13	8	11	5	6	15	0	3	4	7	2	12	1	10	14	9
10	6	9	0	12	11	7	13	15	1	3	14	5	2	8	4
3	15	0	6	10	1	13	8	9	4	5	11	12	7	2	14

Cont . . .

Tables defining the functions $S_5, \dots, S_8$

S_5

2	12	4	1	7	10	11	6	8	5	3	15	13	0	14	9
14	11	2	12	4	7	13	1	5	0	15	10	3	9	8	6
4	2	1	11	10	13	7	8	15	9	12	5	6	3	0	14
11	8	12	7	1	14	2	13	6	15	0	9	10	4	5	3

S_6

12	1	10	15	9	2	6	8	0	13	3	4	14	7	5	11
10	15	4	2	7	12	9	5	6	1	13	14	0	11	3	8
9	14	15	5	2	8	12	3	7	0	4	10	1	13	11	6
4	3	2	12	9	5	15	10	11	14	1	7	6	0	8	13

S_7

4	11	2	14	15	0	8	13	3	12	9	7	5	10	6	1
13	0	11	7	4	9	1	10	14	3	5	12	2	15	8	6
1	4	11	13	12	3	7	14	10	15	6	8	0	5	9	2
6	11	13	8	1	4	10	7	9	5	0	15	14	2	3	12

S_8

13	2	8	4	6	15	11	1	10	9	3	14	5	0	12	7
1	15	13	8	10	3	7	4	12	5	6	11	0	14	9	2
7	11	4	1	9	12	14	2	0	6	10	13	15	3	5	8
2	1	14	7	4	10	8	13	15	12	9	0	3	5	6	11

Cont...

- If S_i is the S-box function table and B is a block of 6 bits, then $S_i(B)$ is determined as follows:
 - The first and last bits of B represent in base 2, a number in the decimal range 0 to 3 (or binary 00 to 11). Let that number be i .
 - The middle 4 bits of B represent in base 2 a number in the decimal range 0 to 15 (binary 0000 to 1111). Let that number be j .
 - Look up in the table the number in the i -th row and j -th column. It is a number in the range 0 to 15 and is uniquely represented by a 4 bit block. That block is the output $S_i(B)$ of S_i for the input B .
- E.g. 1) for input block $B = 011011$ first bit is "0" and last bit "1" giving 01 as row. This is row 1. The middle four bits are "1101". This is the binary equivalent of decimal 13, so the column is number 13. In row 1, column 13 appears 5. This determines the output; 5 is binary 0101, so that the output is 0101. Hence $S_i(011011) = 0101$.

Cont...

- **Example 2)** if the 6-bit number is 100101,
 - Then the first and last bit is 11 and the decimal equivalent of the number is 3.
 - Remaining bits are 0010 and the decimal equivalent of the number is 2.
 - If it is the first block of input, then check the 3rd row 2nd column value in the S-box-1 substitution table. It is given as 1 in the table. Binary equivalent of 1 is 0001.
 - The input 100101 of 6-bit is now reduced to 0001 after S-Box Substitution.
- Accordingly for the 1st round, we obtain as the output of the 8 S boxes

$$K_1 + \mathbf{E}(R_0) = 011000 \ 010001 \ 011110 \ 111010 \ 100001 \ 100110 \\ 010100 \ 100111.$$

$$S_1(B_1)S_2(B_2)S_3(B_3)S_4(B_4)S_5(B_5)S_6(B_6)S_7(B_7)S_8(B_8) = \\ 0101 \ 1100 \ 1000 \ 0010 \ 1011 \ 0101 \ 1001 \ 0111$$

Cont...

P-Box Permutation

- The 32 bit output of S-boxes is then subjected to the straight permutation with rule shown in the following illustration

16	07	20	21	29	12	28	17
01	15	23	26	05	18	31	10
02	08	24	14	32	27	03	09
19	13	30	06	22	11	04	25

- The permutation **P** yields a 32-bit output from a 32-bit input by involving simple permutation bits of the input block that is replacement of each bit with another bit as specified in the p-Box table, without any expansion or compression.

Cont...

- **Example:** From the output of the eight S boxes:

$$S_1(B_1)S_2(B_2)S_3(B_3)S_4(B_4)S_5(B_5)S_6(B_6)S_7(B_7)S_8(B_8) =$$

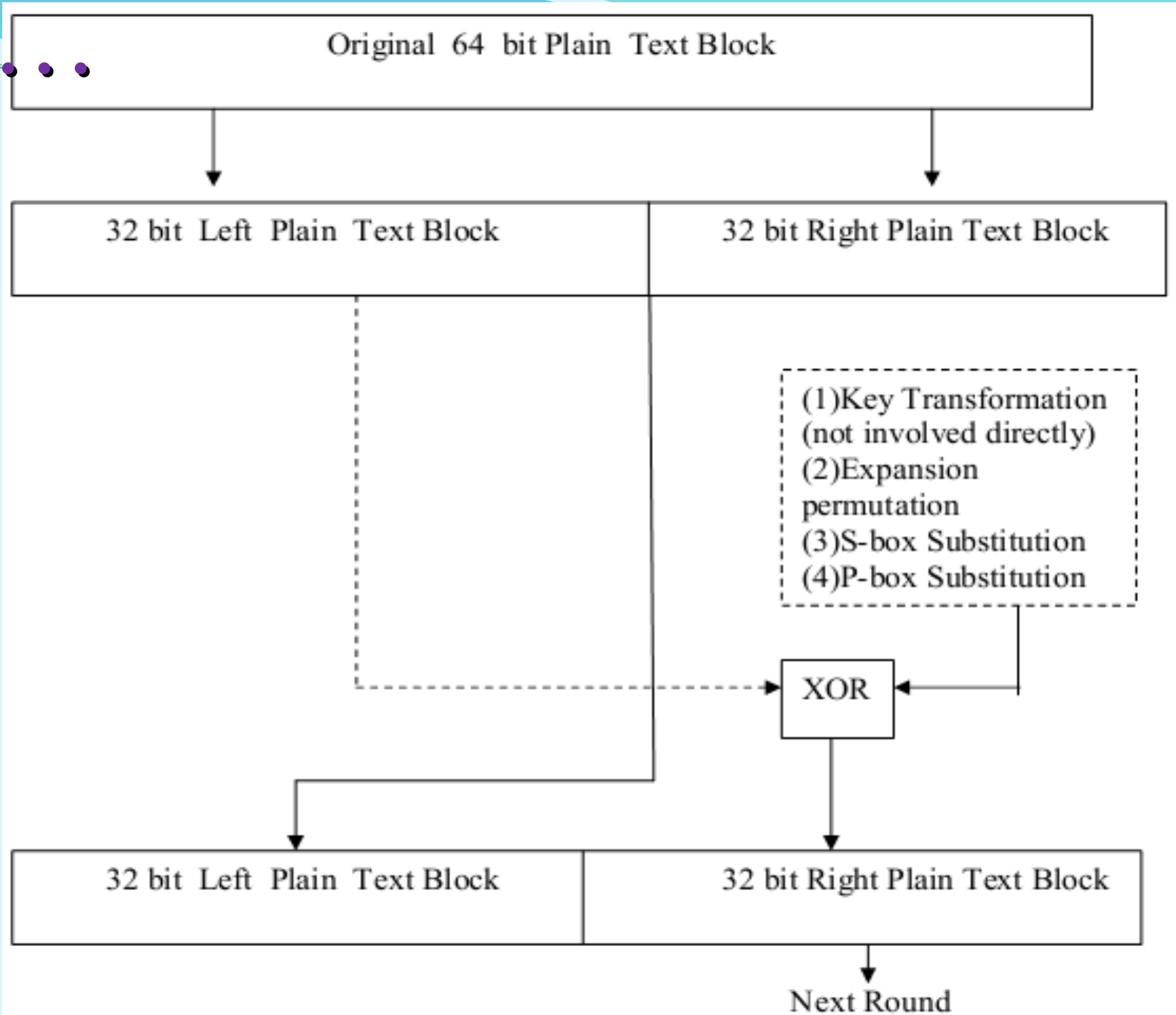
0101 1100 1000 0010 1011 0101 1001 0111

- we get $f = 0010\ 0011\ 0100\ 1010\ 1010\ 1001\ 1011\ 1011$

XOR and Swap

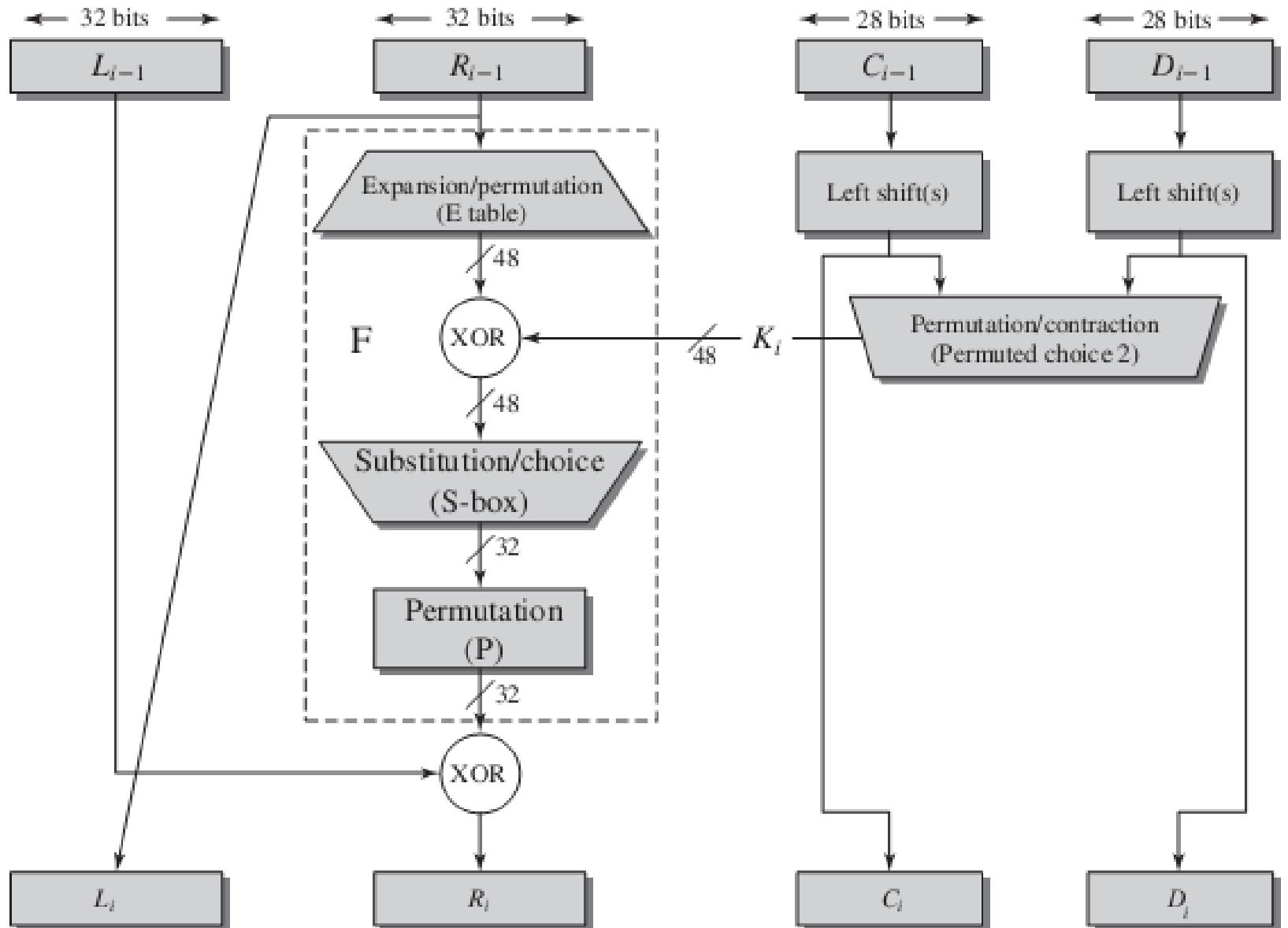
- The untouched LPT, which is of 32 bits, is XORed with the resultant RPT that is with the output produced by P-Box permutation.
- The result of this XOR operation becomes the new right half.
- The old right half becomes the new left half in the process of swapping.
- At end of 16 rounds, the Final Permutation is performed only once.
- This is a simple transposition based on the Final Permutation Table.
- Output of the Final permutation is the 64-bit encrypted block

Cont...



Cont...

Single Round of DES Algorithm



Cont...

Exercise

- Let **M** be the plain text message **M** = 0123456789ABCDEF, where **M** is in hexadecimal (base 16) format. Rewriting **M** in binary format, we get the 64-bit block of text:
- Let **Key** be the hexadecimal key **K** = 133457799BBCDFF1.
- Just setting 1 = 0001, 2 = 0010, 3 = 0011, etc., and grouping together every bits, the equivalent binary representation of **M** and **K** are:

M = 0000 0001 0010 0011 0100 0101 0110 0111 1000 1001
1010 1011 1100 1101 1110 1111

K = 00010011 00110100 01010111 01111001 10011011
10111100 11011111 11110001

- What is the output ciphertext of DES of given plaintext M?**

DES Analysis

- The DES satisfies both the desired properties of block cipher which make cipher very strong.

Avalanche effect

- A desirable property of any encryption algorithm is that a small change in either the plaintext or the key should produce a significant change in the ciphertext.
- In particular, a change in one bit of the plaintext or one bit of the key should produce a change in many bits of the ciphertext. This is referred to as the *avalanche effect*.
- If the change were small, this might provide a way to reduce the size of the plaintext or key space to be searched

Completeness: Each bit of ciphertext depends on many bits of plaintext

DES Strength (what makes DES strong?)

- *The Use of 56-Bit Keys*

- With a key length of 56 bits, there are 2^{56} possible keys, which is approximately 7.2×10^{16} keys. Thus, on the face of it, a brute-force attack appears impractical.
- Assuming that, on average, half key space has to be searched, a single machine performing one DES encryption per microsecond would take more than a thousand years to break the cipher.
- **Challenges:** *parallel machine and key-search attack*
- But DES finally and definitively proved insecure in July 1998, when the Electronic Frontier Foundation (EFF) announced that it had broken a DES encryption using a special-purpose “DES cracker” machine that was built for less than \$250,000.

Cont...

- ***The Nature of the DES Algorithm***

- Because the design criteria for S-boxes, and indeed for entire algorithm, weren't made public, there is suspicion that S-boxes were constructed in such a way that cryptanalysis is possible for an opponent who knows the weaknesses in the S-boxes.

- ***Timing Attacks***

- Information about the key or plaintext is obtained by observing how long it takes given implementation to perform decryptions on various ciphertexts.
- It exploits the fact that an en/decryption algorithm often takes slightly different amounts of time on different inputs.
- DES appears to be fairly resistant to a successful timing attack

Multiple Encryption with DES

- You can improve the security of DES by performing multiple encryptions, known as super encryption.
- The two most common ways of doing this are with double encryption (Double DES) and with triple encryption (Triple DES).

(1) Double DES

- Each 64-bit block of data is encrypted twice with the DES algorithm, first with one key, then with another, as follows:
 1. Encrypt with (key 1).
 2. Encrypt with (key 2).

Plaintext **(key1)** **(key2)** **ciphertext:**

————> ————> ————>

Cont...

Problem with Double DES

(1) Issue of reduction to single stage

- Suppose it were true for DES, for all 56-bit key values, that given any two keys K_1 and K_2 , it would be possible to find a key such that $E(K_2, E(K_1, P)) = E(K_3, P)$
- If this were the case, then double encryption, and indeed any number of stages of multiple encryption with DES, would be useless because the result would be equivalent to a single encryption with a single 56-bit key.
- Reasonably, if DES is used twice with different keys, it will produce one of the many mappings that are not defined by a single application of DES

Cont...

(2) Meet-in-the-middle attack

- Does not depend on any particular property of DES but that will work against any block encryption cipher.
- A known plaintext attack which requires that an attacker have both a known piece of plaintext and a block of that same text that has been encrypted
- Since $X = E_{K_1}(P) = D_{K_2}(C)$
- Given known pair, (P, C) , attack proceeds as follows. First, encrypt for all possible values of 2^{56} . Store these results in a table and then sort the table by the values of X . Next, decrypt using all possible values of 2^{56} . As each decryption is produced, check the result against the table for a match. If a match occurs, then test the two resulting keys against a new known plaintext–ciphertext pair. If the two keys produce correct ciphertext, accept them as correct keys.

Cont...

(2) Triple DES with Two Keys

- It is just a DES with two 56-bit keys applied.
- Given plaintext, 1st key is used to DES- encrypt the message.
- 2nd key is used to DES-decrypt the encrypted message. (Since 2nd key is not right key, this decryption just scrambles the data further.)
- The twice-scrambled message is then encrypted again with the first key to yield the final ciphertext.

$$C = E(K_1, D(K_2, E(K_1, P)))$$

$$P = D(K_1, E(K_2, D(K_1, C)))$$

(3) Triple DES with Three Keys

- Triple-DES can also done with 3 separate keys instead of only two
- Three-key 3DES has an effective key length of 168 bits and is defined as

$$C = E(K_3, D(K_2, E(K_1, P)))$$

DES Crypto Analysis

- Given a set of known plaintexts and crypto texts, it is possible to analyze the pairs and construct and reduce the number of keys it is necessary to check.
- Examples of such an approach are:

1. Differential crypto-analysis

- The rationale behind differential cryptanalysis is to observe the behavior of pairs of text blocks evolving along each round of the cipher, instead of observing evolution of a single text block

2. Linear crypto-analysis

- This attack is based on finding linear approximations to describe the transformations performed in DES.

DES Design Criteria

- 7 criteria for S-boxes provide for
 - non-linearity
 - resistance to differential cryptanalysis
 - good confusion
- 3 criteria for permutation P provide for
 - increased diffusion

Reading Assignment
Cryptography and Network Security Principles and Practice
5th Edition --- Page 92 - 96

Reading Assignment

The IDEEA Algorithm

IDEA Algorithm and Its Working

- The IDEA is perceived as one of the strongest cryptographic algorithms. It was launched in 1990 and underwent certain changes in names and capabilities as shown in table.

Year	Name
1990	Proposed Encryption Standard(PES)
1991	Improved Proposed Encryption Standard(IPES)
1992	International Data Encryption Algorithm(IDEA)

- One popular email privacy technology known as Pretty Good Privacy (PGP) is based on IDEA.
- It is block cipher which is reversible like DES, i.e. the same algorithm is used for encryption and decryption.
- It uses both confusion and diffusion for encryption.

Cont...

Algorithm:

Step 1) Consider the input plain text of 64 bits.

Step 2) Divide the input plain text into 4 portions each of size 16 bits (Say P_1 to P_4).

Step 3) Now perform 8 rounds of algorithm.

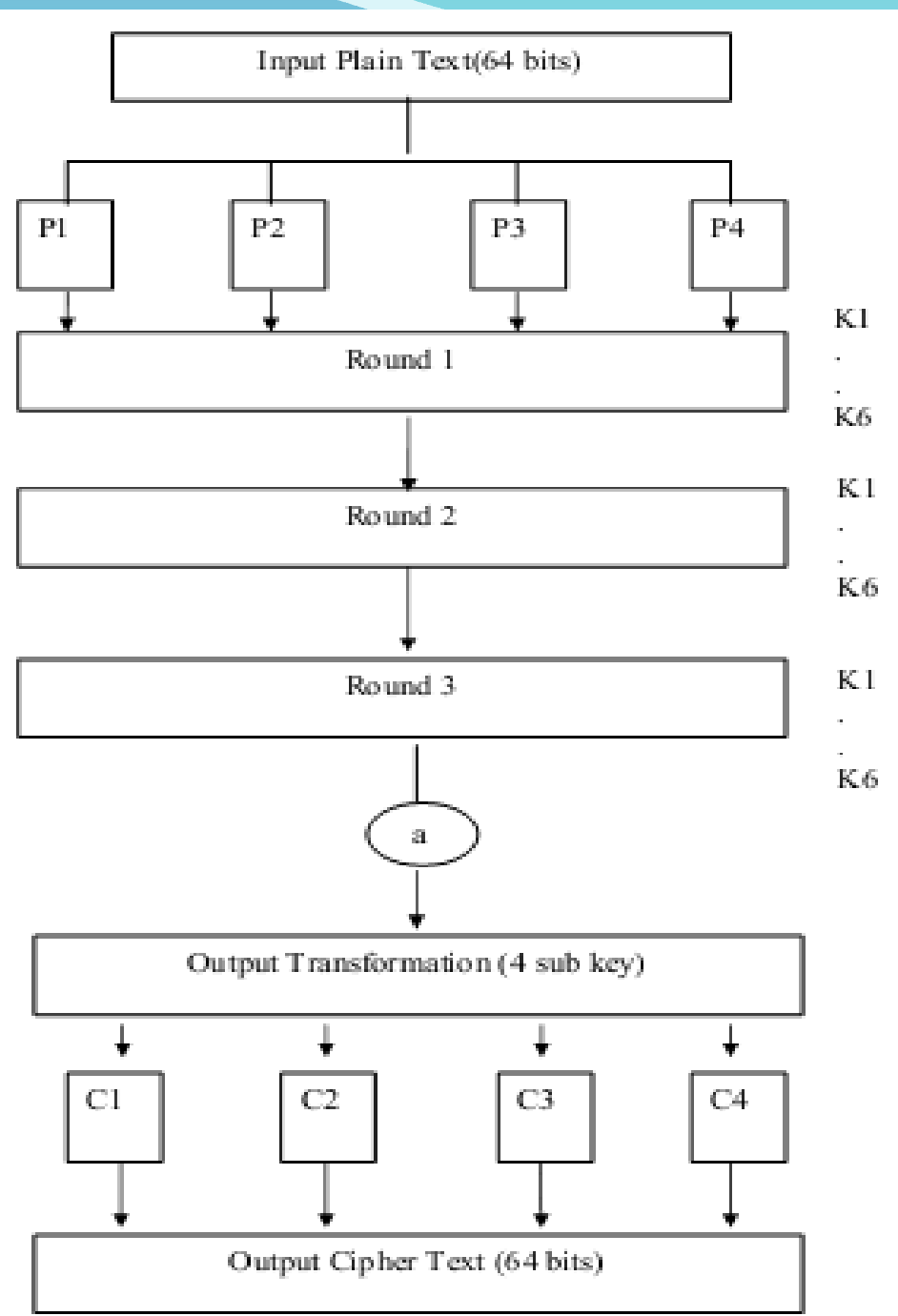
(a) In each round 6 sub-keys are generated from the original key. Each of the sub-keys consists of 16-bits. These six sub-keys are applied to four input blocks P_1 to P_4 . Thus for first round, we have 6 keys say k_1 to k_6 ; for second round, we have k_7 to k_{12} . Finally for eight round we have keys k_{43} to k_{48} .

(b) Multiply, add and XOR the plain text blocks with sub keys.

Step 4) Perform an output transformation in sub-keys.

Step 5) Combine all 4 blocks of output transformation to get ciphertext

Cont...



Cont...

Details of first round in IDEA

- The initial key consists of 128 bits from which 6 sub-keys k_1 to k_6 are generated for the first round.
- Since k_1 to k_6 consist of 16 bits each, out of the original 128 bits, the first 96 bits ($6 * 16$ bits per sub-key) are used for first round.
- Thus at end of 1st round, bits 97-128 of original keys are unused.

Details of second round in IDEA

- In 2nd round 31 unused bits are used. For 2nd round we still require ($96 - 31 = 65$) more bits. But the original key 128 bits are exhausted.
- Now IDEA uses the techniques of key shifting and original key is shifted left circularly by 25 bits.

Cont...

Details of One round in IDEA

- 1) Multiply P1 and k1
- 2) Add P2 and K2
- 3) Add P3 and k3
- 4) Multiply P4 and k4
- 5) XOR the results of steps 1 and 3
- 6) XOR the results of steps 2 and 4
- 7) Multiply step 5 and k5
- 8) Add steps 6 and step 7
- 9) Multiply the result of step 8 and k6
- 10) Add steps 7 and 9
- 11) XOR the results of steps 1 and 9
- 12) XOR the results of steps 3 and 9
- 13) XOR the results of steps 2 and 10
- 14) XOR the results of steps 4 and 10

Cont...

Details of output transformation

- (1) It is one time operation. It takes place at end of the 8th round
- (2) It is 64 bit value divided into 4 sub-blocks (say R1 to R4 each consisting of 16 bits).

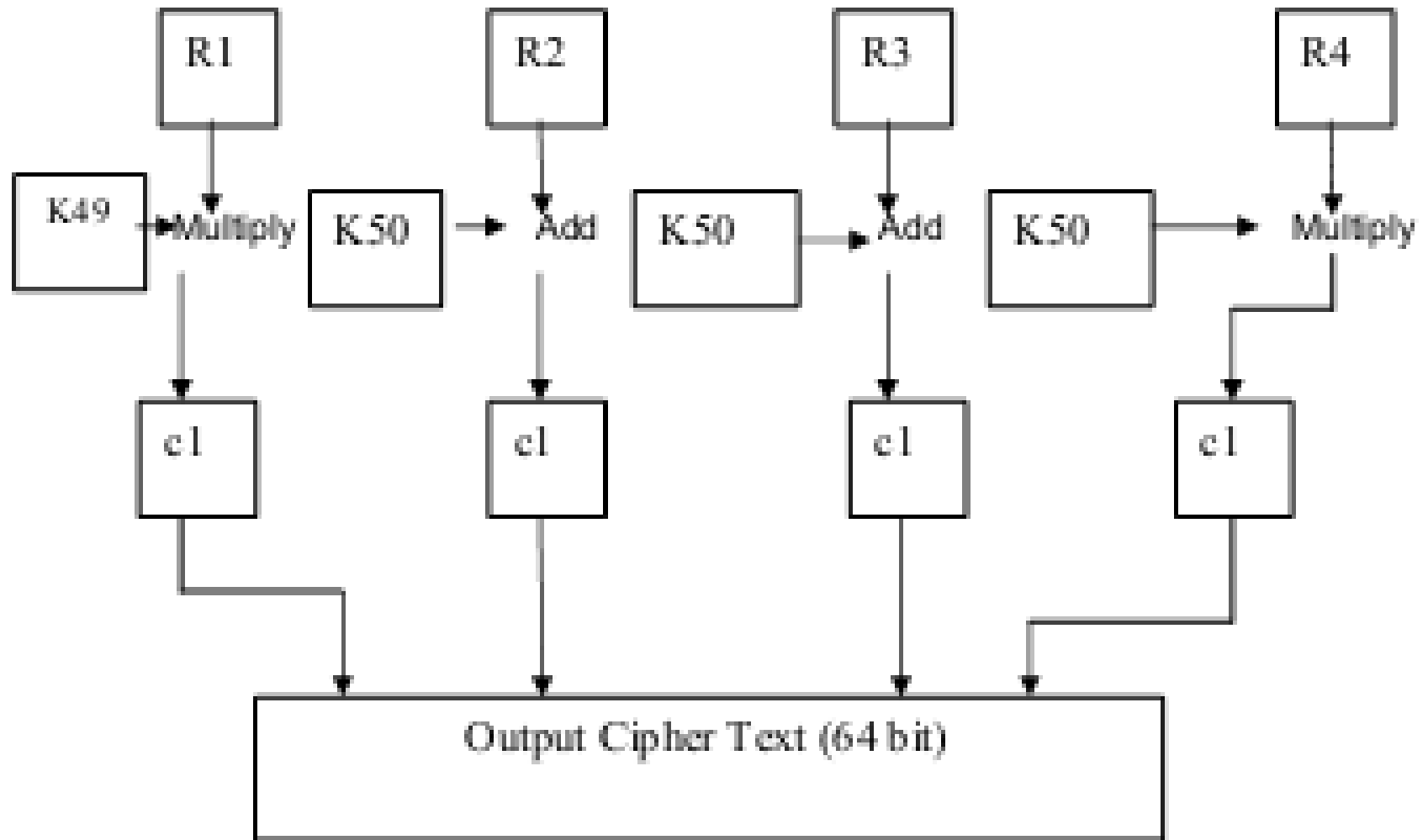
Step 1: Multiply R1 and k49

Step2: Add R2 and k50.

Step3: Add r3 and k51

Step4: Multiply r4 & k52

Cont...



Cont...

A Symmetric Cryptosystems Comparison Table

Cipher	Security	Speed (486 pc)	Key length
DES	low	400 kb/s	56 bits
Triple DES	good	150 kb/s	112 bits
IDEA	good*	200 kb/s	128 bits
Triple IDEA	very good*	~ 100 kb/s	256 bits



THE END

???